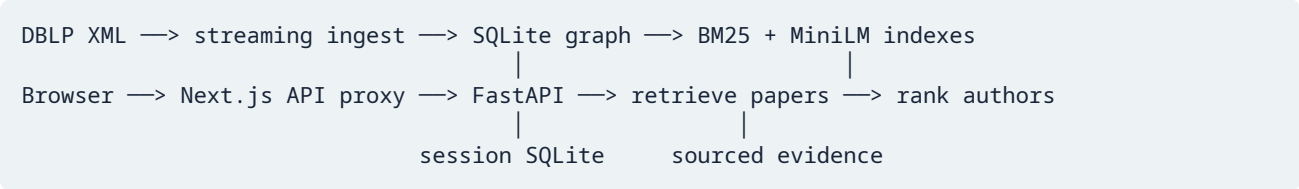
ScholarGraph — design document

ScholarGraph finds researchers in DBLP and presents the publications and relationships behind each result. The reviewer can run the bundled sample with Docker Compose or use the offline batch interface on another DBLP XML slice. The application uses Next.js/TypeScript, FastAPI/Python, SQLite and local ONNX models; no API key is required after provisioning.

Architecture and graph

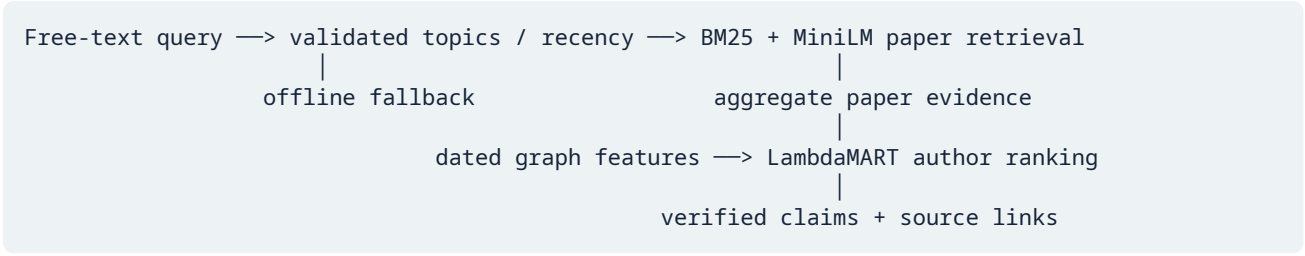


The graph has person, paper and venue nodes; authorship, author–venue and coauthorship edges carry a publication year and DBLP source key. Re-ingestion is idempotent. Serving opens the graph read-only and keeps mutable sessions in a separate SQLite database. DBLP PIDs distinguish authors where supplied; otherwise ingestion falls back to names. Public DBLP data is CC0.

An experimental noisy-author linker blocks on name variants and coauthors before scoring names, coauthors, venues and years. On synthetic known-reference linkage it measured 98.25% precision and 16.76% known-author recall; it is not the serving identity pipeline or open-world clustering. Incorrect merges cost more than splits because they attach another person's evidence to a profile. Conservative matching and abstention reduce that risk.

SQLite and local indexes were chosen for portable, keyless operation and a small operational footprint. PostgreSQL plus a separate search service would offer replication and concurrency at the cost of more moving parts. The reference application is not a multi-tenant service.

Retrieval, learning and measurement



BM25 and a pretrained local MiniLM encoder each retrieve up to 1,024 papers; evidence is aggregated into at most 800 author candidates. A project-trained 80-tree LambdaMART model orders candidates using retrieval, publication, recency and graph features. Scores are ranking signals, not probabilities. A lexical-overlap guard reduces unsupported matches but can also suppress valid synonym-only queries.

Training labels use later paper titles as queries and their authors as proxy positives against earlier graph snapshots. Training history ends in 2020, validation history in 2023, and the 500-query historical test uses 2025 titles against a graph through 2024. Source IDs, dates and exact normalized titles are checked for leakage; semantic near-duplicates and identity errors remain possible. The test has been inspected and must not be treated as a new untouched test.

Method	nDCG@10	MRR (up to 800)
BM25 lexical	0.0245	0.0338
Pretrained dense	0.0268	0.0361
Untrained hybrid	0.0313	0.0453
Trained LambdaMART	0.0354	0.0488
Without explicit recency	0.0334	0.0479

Future authorship is incomplete relevance supervision; other authors are unjudged. Absolute scores are low and do not measure hiring quality. The recency ablation keeps retrieval fixed and other features may retain temporal signals. A name-swap check changed none of eight rankings; this is not proof of fairness. A fresh index replay shifted dense-based scores; rescoring preserved historical features reproduced the reported numbers. The results file records both, failure cases and replay instructions.

Assistant, safety and operations

The API exposes a command-based graph assistant with three schema-validated tools: search experts, retrieve a profile and find a collaboration path. Its session memory records shortlist and reject decisions, and typed factual claims are checked against papers and edges. The shared conversation panel can attach a selected researcher profile, but does not currently expose the graph assistant tools or applied memory. The search UI supports shortlist, rejection and memory deletion. In a historical scripted benchmark, 10 scenarios passed, 0 of 191 typed claims were unsupported, and 8 of 8 scripted query attacks were blocked. These bounded results do not verify general generated prose or guarantee injection resistance.

Query understanding supports a schema-validated local model, with recorded and deterministic fallback. Paper text and model output cannot select graph tools. The shared conversation can use a remote provider when configured; an explicitly attached profile is passed as untrusted context. General conversation prose is not covered by the typed-claim verifier. The API exposes health and request logs. Tests, browser checks and a Compose smoke check run in CI. Building images downloads dependencies and models; provisioned runtime and batch execution need no API key.

Scaling and privacy

For a target of 50 million papers, 200 queries/s and p95 500 ms, partition ingestion and store versioned graph snapshots; precompute embeddings and author features; shard and replicate lexical and approximate-vector indexes; bound reranking and batch evidence reads; add caching, backpressure and monitoring. An illustrative budget is 30 ms network/auth, 150 ms retrieval, 100 ms reranking, 100 ms evidence and 120 ms other overhead. These are design targets, not measurements; validate ANN recall, update lag, cost, resilience and concurrency before committing to them.

DBLP records are public, but profiles and shortlist decisions still require access control, tenant isolation, retention/deletion rules and audit logs. Versioned, dated edges can extend the context graph. Outreach should require human approval; feedback used for learning should be consented, bias-checked and evaluated separately. The current demo has no accounts or tenant isolation.